

K-Nearest Neighbours

Classify anything by asking:

"What do my nearest neighbours look like?"

What You Will Learn Today

By the end of this lecture you can **explain, apply, and critically evaluate** the KNN algorithm.

01 Explain how KNN classifies a new data point using the majority vote of its K nearest labelled neighbours

02 Compute Euclidean distance — the foundation of KNN

03 Choose k systematically using a validation curve

04 Explain why feature scaling is non-negotiable for KNN

05 Recognise the curse of dimensionality and its impact

06 Decide when to use KNN versus Logistic Regression

The core idea is simple: look at your neighbours.

No equations during training. No coefficients to fit. Just remember the data — and ask the nearest points what class they belong to.

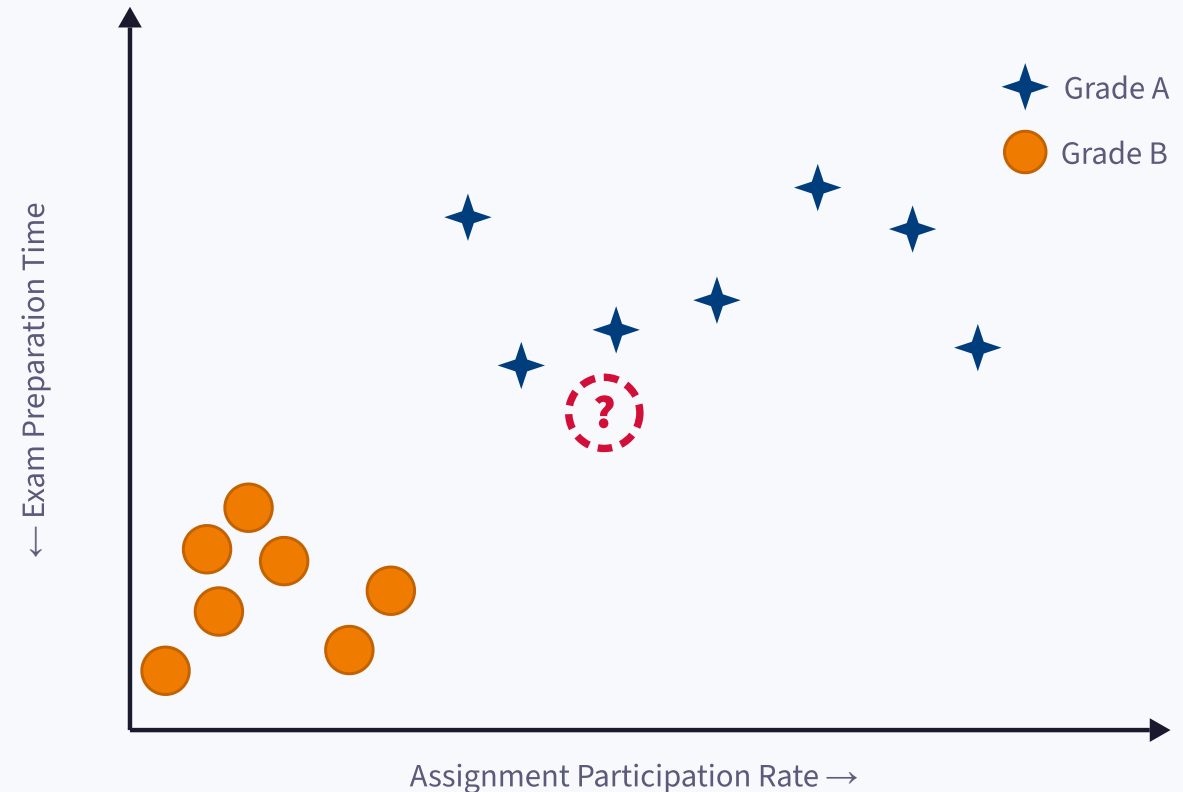
The Problem: Predicting a Student's Grade

Each student has two **features** and one **label**:

- ▶ **Feature 1** — Exam Preparation Time
- ▶ **Feature 2** — Assignment Participation Rate
- ▶ **Label** — Grade A (above avg) or Grade B (below avg)

A new student **"?"** arrives with known features.

We want to predict: **Grade A or Grade B?**



New student **"?"** — Grade A or Grade B?

KNN's Approach: Think Locally, Not Globally

The KNN idea: Find the k training students closest to the new student in feature space. Let them **vote**. The majority class wins.

What KNN does NOT do

- ▶ Learn a global formula during training
- ▶ Fit any coefficients or weights
- ▶ Draw a global decision boundary

What KNN DOES do

- ▶ Memorise all training examples as-is
- ▶ At prediction: find nearest neighbours
- ▶ Return the majority class of those neighbours

💡 This is called a **lazy learner** — no training work happens upfront. All computation is deferred to prediction time.

Four steps. That is the whole algorithm.

KNN is one of the simplest ML algorithms to explain.

Understanding each step also reveals its key limitations.

Step 1 — Measure Distance to Every Training Point

KNN doesn't see "students" — it sees **coordinates**. To find neighbours it measures the **straight-line distance** between points.

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

i Variables:

x_1, y_1 = coordinates of the *training point*

x_2, y_2 = coordinates of the *new point* "?"

i Why it works:

this is the Pythagorean theorem — the hypotenuse of a right triangle whose legs are the feature differences.

It generalises to n features: add more squared terms under $\sqrt{\dots}$.

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{j=1}^n (q_j - p_j)^2}$$

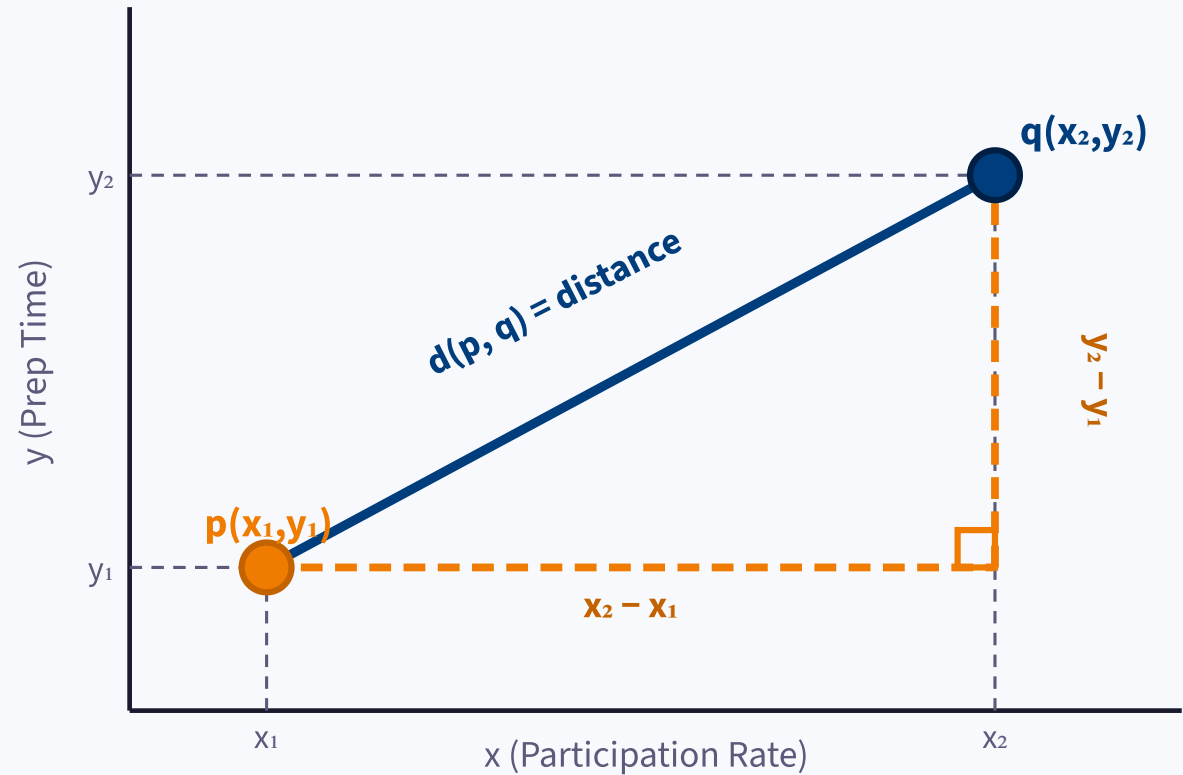
Distance Visualised — The Pythagorean Triangle

The distance $d(p, q)$ is the length of the **hypotenuse**.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- ▶ Horizontal leg = $x_2 - x_1$
- ▶ Vertical leg = $y_2 - y_1$
- ▶ Hypotenuse = $d(p, q)$

💡 The right-angle marker shows where the two legs meet. The blue line is the actual Euclidean distance.



Euclidean distance = length of the hypotenuse of the right triangle

Steps 2, 3 and 4 — Sort, Select, Vote

② **Sort** all distances ascending. Keep the k smallest — these are the k nearest neighbours.

③ **Look up** the class label of each of those k neighbours in the training data.

④ **Majority vote** — the most common class among the k neighbours is the prediction.



Classification: majority vote chooses the most common class label among the k nearest neighbours.

Worked Example — $k = 5$

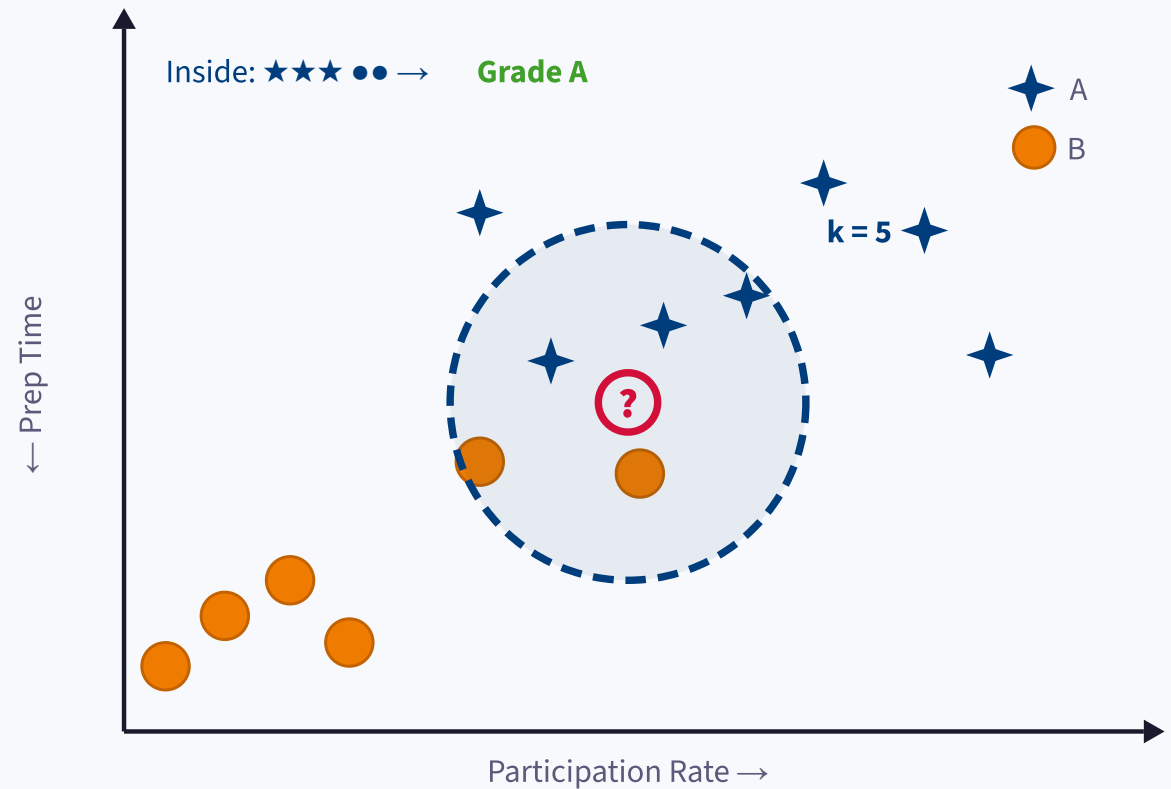
Set $k = 5$. The dashed circle captures exactly the **5 nearest neighbours** of "?".

✓ 3 neighbours are **Grade A** ★

✓ 2 neighbours are **Grade B** ●

→ **Majority: Grade A wins (3 vs 2)**

Prediction for "?" = **Grade A**



k=5 circle: 3 Grade-A stars, 2 Grade-B circles →
predict A

KNN Regression — Predicting a Numeric Value

Same idea, different target.

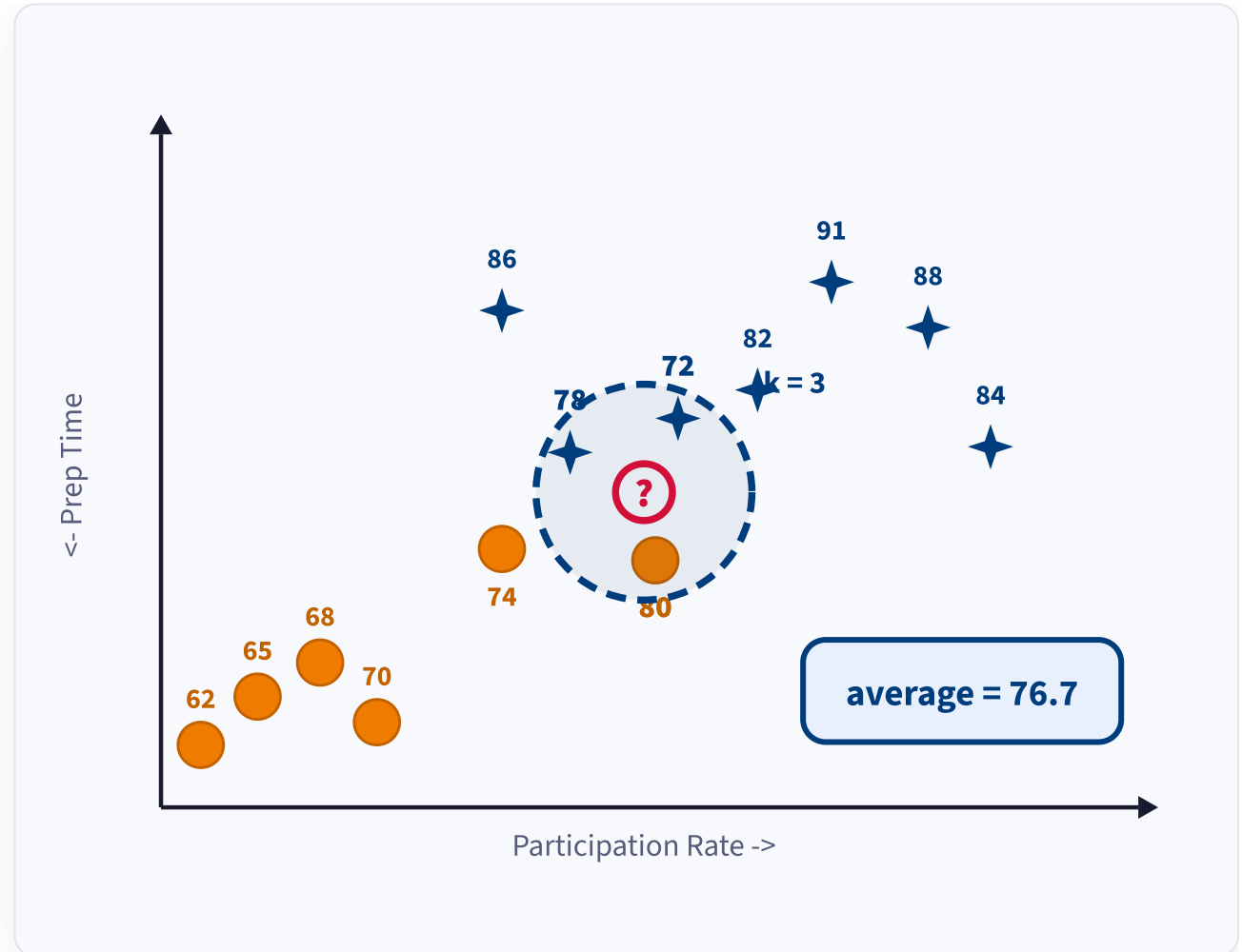
Instead of predicting Grade A or B, predict a numeric outcome such as final score.

1 Find the k nearest students

$$\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k} y_i$$

Nearest 3 students scored **72**, **78**, and **80**.

Predicted score = $\frac{72 + 78 + 80}{3} = 76.7$



i Classification uses majority vote. Regression uses averaging.

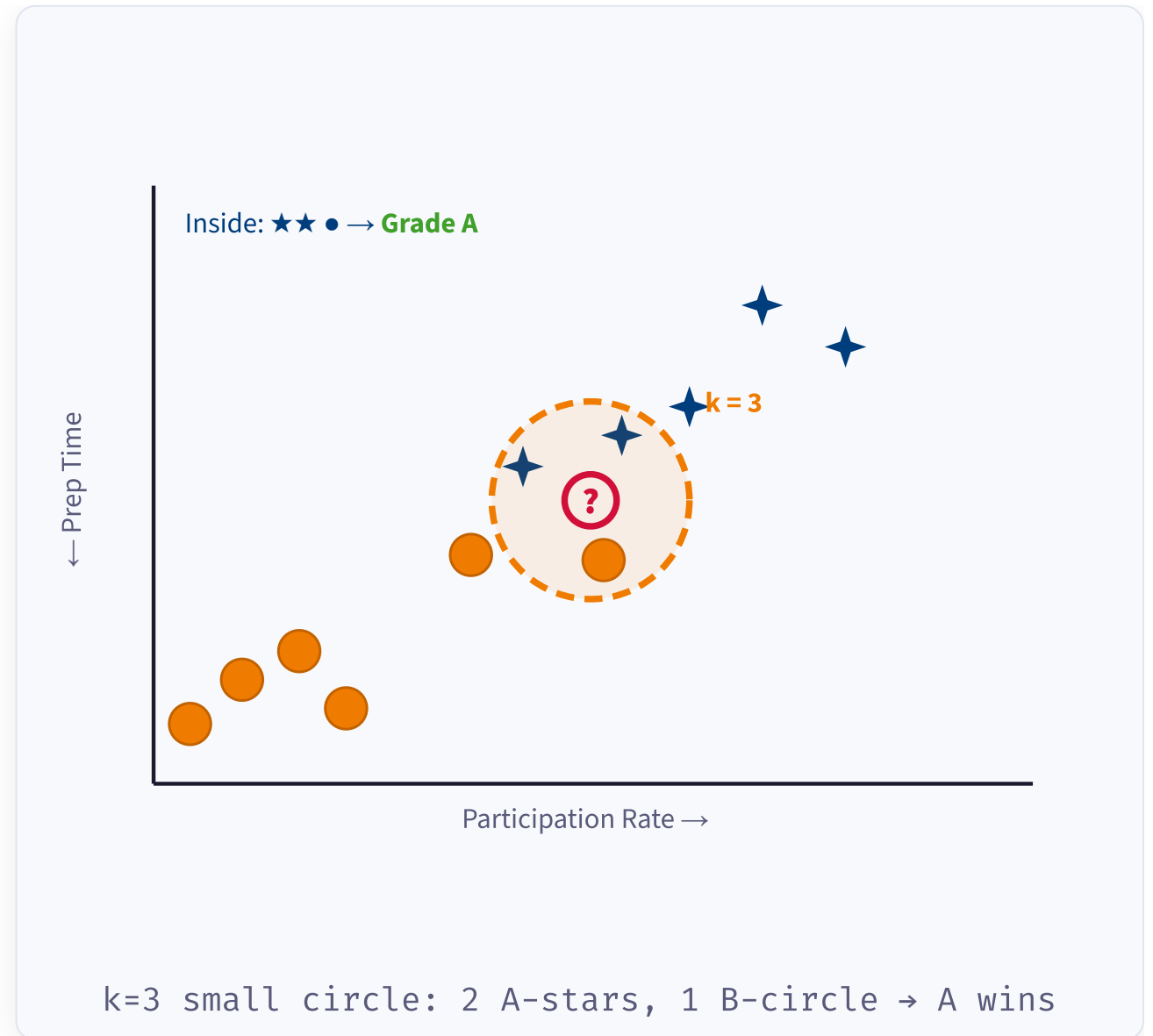
The value of k changes the prediction.

Different values of k can classify the same point differently — sometimes completely reversing the result.

$k = 3 \rightarrow$ Predicts Grade A

With a **small circle** ($k = 3$), we capture only the 3 nearest neighbours.

- ✓ 2 = **Grade A** ★
- ✓ 1 = **Grade B** ●
- **Vote: Grade A (2 vs 1)**
- $k = 3$ prediction: **Grade A**



$k = 7 \rightarrow$ Predicts Grade B — The Prediction Flipped!

Widening to $k = 7$ pulls in more neighbours and the balance shifts.

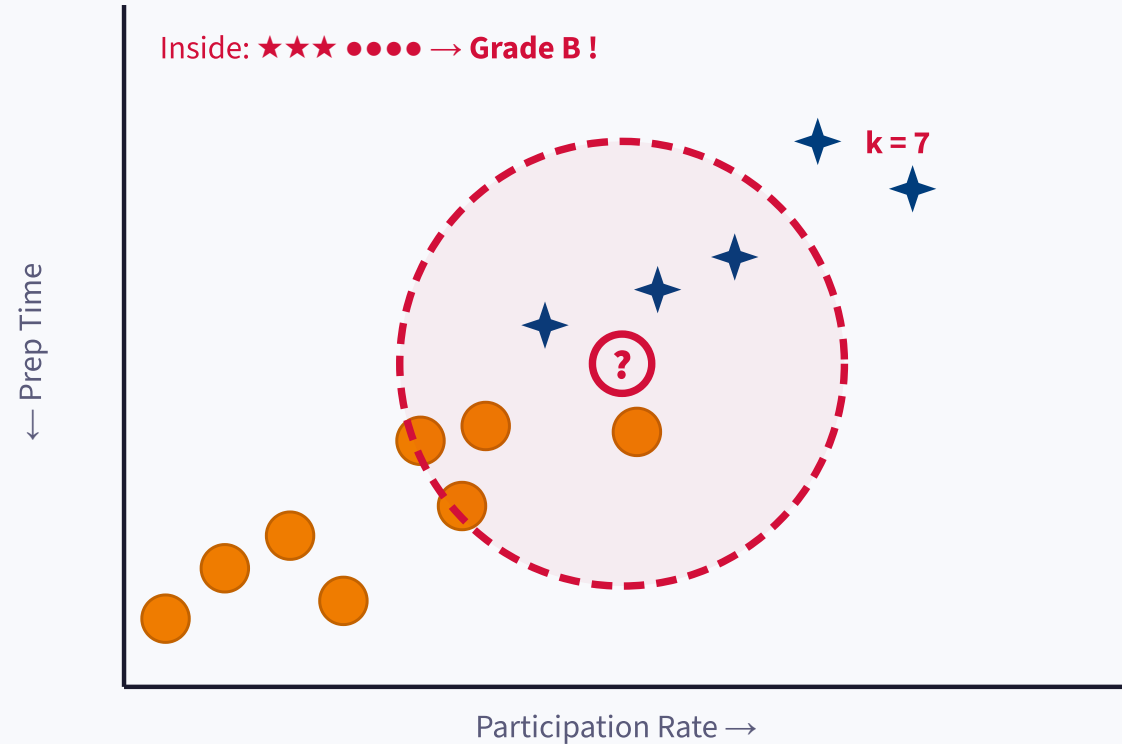
✓ 3 = **Grade A** ★

✓ 4 = **Grade B** ●

→ **Vote: Grade B wins (4 vs 3)!**

Same point. Same data. Different k .

Completely different prediction. Choosing k is the most important design decision in KNN.



k=7 larger circle: 3 A-stars, 4 B-circles → B wins (flipped!)

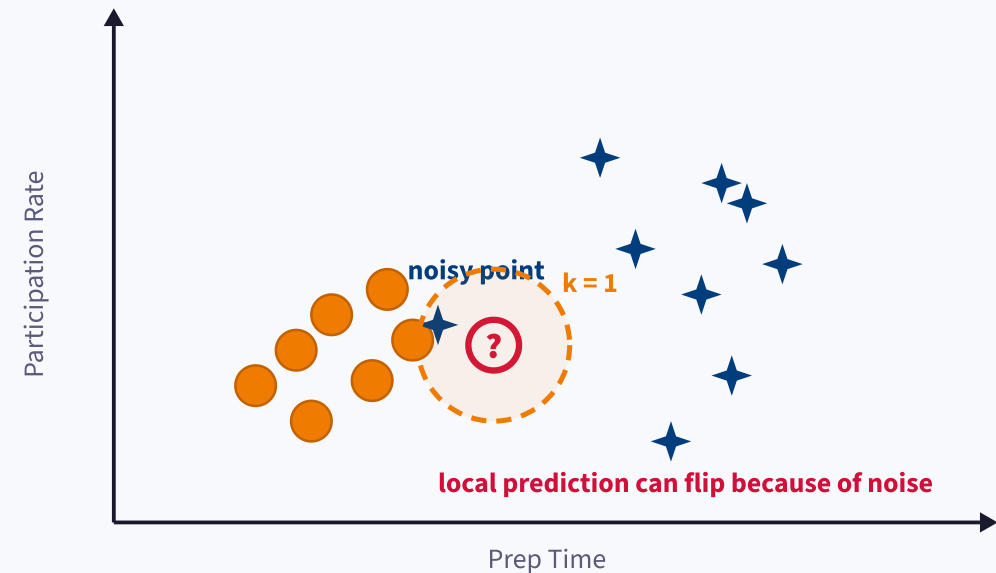
k Too Small: The Model Overfits

When $k = 1$, one neighbour has too much power

With $k = 1$

- ▶ Only one neighbour decides the prediction
- ▶ If that point is noisy or unusual, the prediction can be wrong
- ▶ Small changes in position can flip the result

Takeaway: the model memorises individual points instead of learning the broader pattern.



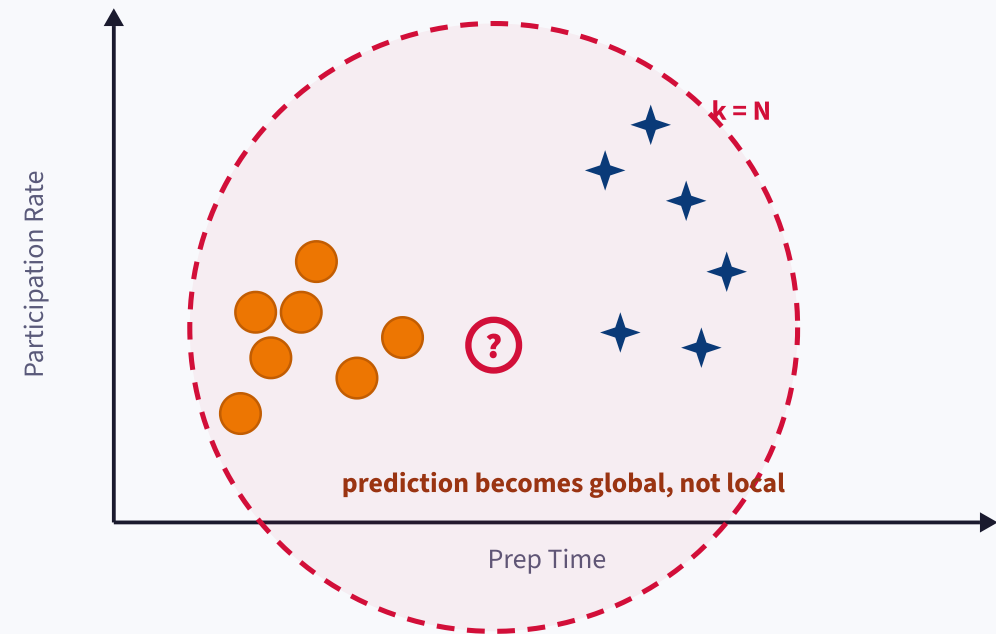
k Too Large: The Model Underfits

When k is too large, the neighbourhood becomes too broad

With $k = N$

- ▶ Every training point becomes a neighbour
- ▶ The model predicts the overall majority class
- ▶ The location of "?" no longer matters much

Takeaway: the model oversmooths the data and misses the local pattern.

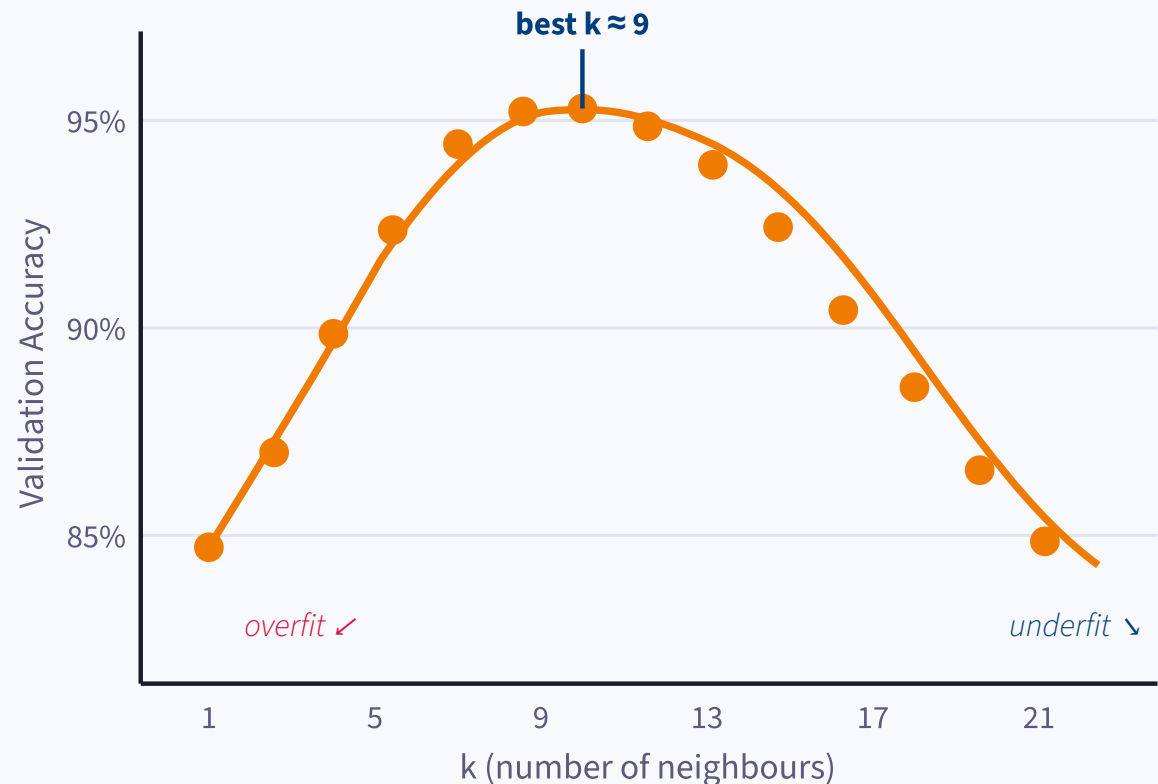


How to Choose k : The Validation Curve

Use the train/test split from Lecture 04.
For each candidate k , measure accuracy on the validation set.

- ① Try $k = 1, 3, 5, \dots, 25$ (odd values)
- ② Record validation accuracy for each k
- ③ Pick the k at the **peak** of the curve

i Use F1 when classes are imbalanced. If 95% of students get Grade B, accuracy = 95% for a model that always predicts B. F1 catches this by combining precision and recall.



Validation accuracy peaks at the sweet-spot k value

Three Practical Rules for Choosing k

- ① **Always prefer odd k** for binary classification. An even k risks a tie vote (e.g. 2 A vs 2 B), producing no clear winner.
- ② **Start with $k \approx \sqrt{N}$** as an initial guess, where N is the number of training samples. For $N = 400$ training points, try $k \approx 20$ first.
- ③ **Always measure.** Plot the validation curve and pick the k that maximises validation accuracy or F1. Never trust a rule of thumb over a measurement.

💡 The right k is **data-dependent**. There is no universally correct value — only the value that works best on your specific dataset.

Two traps that will silently break your model.

Neither produces an error message. Both lead to wrong predictions without warning — unless you know what to look for.

Pitfall 1 — Feature Scaling

KNN is entirely based on **distance**. When features have very different value ranges, the feature with the **largest range dominates** every distance calculation — making all other features irrelevant.

Without scaling

- ▶ Feature A ranges 0 – 200,000
- ▶ Feature B ranges 0 – 30

Feature A controls 99.99% of every distance calculation. Feature B might as well not exist.

After z-score scaling

- ▶ Both features: mean = 0, std = 1
- ▶ Each contributes equally and fairly

Distance now reflects relative difference, not absolute scale.

Rule: Always apply `StandardScaler` before fitting KNN. Fit on training data only — never refit on test data.

Scaling: The Problem — Loan Approval Example

Two features: **Annual Salary** (range: \$20k – \$200k) and **Years of Experience** (range: 0 – 30). Ratio = $\frac{180,000}{30} = 6,000$ to 1.

Person	Annual Salary	Years of Experience
A	\$50,000	5 years
B	\$52,000	28 years
C	\$180,000	6 years

Distance A → B (no scaling)

$$d = \sqrt{2000^2 + 23^2} \approx 2,000$$

Model thinks A and B are close — but B has 23 more years of experience!

Distance A → C (no scaling)

$$d = \sqrt{130000^2 + 1^2} \approx 130,000$$

Salary gap of \$130k completely swamps the 1-year experience gap.

Only salary matters. Experience is invisible.

Scaling: The Fix — Z-Score Standardisation

Definition of z-score

$$z = \frac{x - \mu}{\sigma}$$

- ▶ μ = mean of the feature across all training samples
- ▶ σ = standard deviation
- ▶ $z = 0$ means exactly average
- ▶ $z = +1$ means one std deviation above average

i After z-scoring: **every feature has $\mu = 0$ and $\sigma = 1$** . All features live on the same scale.

After scaling — same three people

Person	Salary (z)	Experience (z)
A	-0.83	-0.80
B	-0.80	+1.50
C	+1.33	-0.70

Result after z-score scaling

$d(A, B) \approx 2.3$ — experience gap is now clearly visible

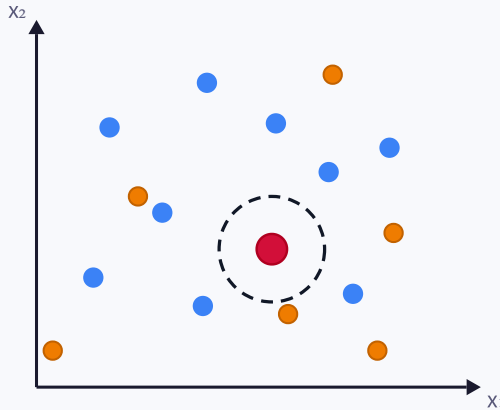
$d(A, C) \approx 2.2$ — salary gap properly weighted

Both features contribute meaningfully.

Pitfall 2 — The Curse of Dimensionality

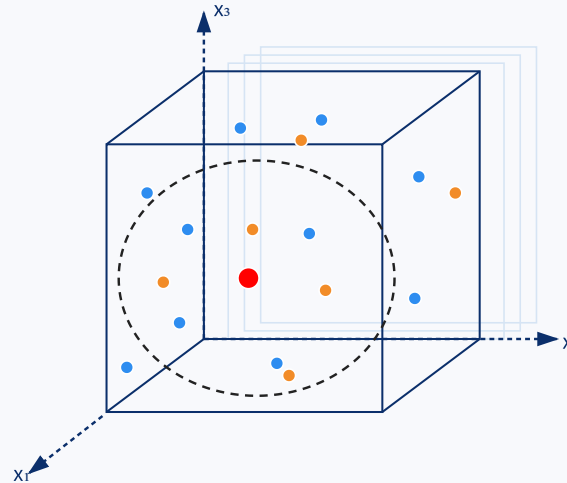
As the number of features increases, the space grows **extremely fast**. With the same amount of training data, points become sparse, so even the nearest neighbours may still be far away.

1) Low dimension



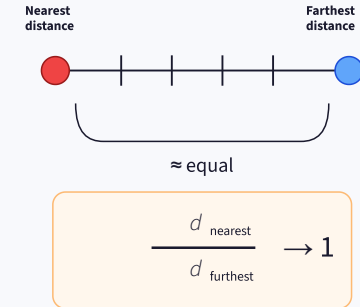
Nearby points are truly near

2) Higher dimension



Points become sparse

3) What happens to KNN?



- ▶ All points are at similar distances
- ▶ Majority vote becomes less meaningful



**How to
reduce the
problem**

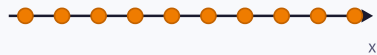
- ✓ Remove irrelevant features
- ✓ Keep only informative dimensions before KNN

- ✓ Use feature selection or PCA

The Curse: Exponential Data Requirements

Simple idea: suppose each feature has **10 possible levels**. Then every new feature makes the data requirement **10 times bigger**.

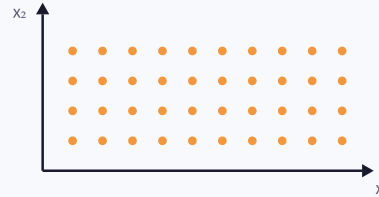
1 Feature



One feature gives a line

$$10^1 = 10$$

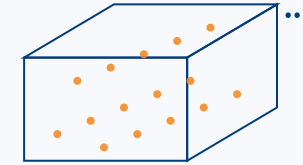
2 Features



Two features give a grid

$$10^2 = 100$$

3 Features



Three features give a cube

$$10^3 = 1,000$$

What these three diagrams mean

As we add more features, the data must spread across a larger space.

Each extra feature creates many more places that need data.

i Beginner takeaway: if 3 features need about **1,000** points, then 5 features need **100,000**, and 10 features need **10 billion**. KNN struggles because the space grows faster than the data.

You now have two classifiers. When do you use which?

Logistic Regression (Lecture 04) and KNN solve the same problem in fundamentally different ways. A plain-English comparison follows.

KNN vs Logistic Regression — Training & Prediction Speed

KNN — Lazy Learner

- ▶ **Training:** store the training data
- ▶ **Prediction:** compare the new point with all N training points
- ▶ Each comparison uses d features, so prediction is $O(Nd)$

Logistic Regression — Eager Learner

- ▶ **Training:** learn the weights by optimization
- ▶ **Prediction:** compute one weighted sum across the d features
- ▶ So prediction stays small: $O(d)$

i Example: if the bank stores more and more past customers, KNN must compare a new case with more and more records. Logistic Regression still uses the same learned formula, so prediction stays fast.

KNN vs Logistic Regression — Decision Boundary Shape

KNN — Flexible, Locally Adaptive

- ▶ The boundary follows the **local shape** of the data — curved, complex, non-linear
- ▶ Can wrap around clusters, create islands and rings
- ▶ Great when classes are **not linearly separable**

Logistic Regression — Always a Straight Line

- ▶ The boundary is always a **hyperplane** (straight line in 2D)
- ▶ Cannot capture circular, spiral, or ring-shaped distributions
- ▶ Great when classes are **approximately linearly separable**

Choose KNN when

The data has a non-linear, complex shape; small dataset; no need for interpretability.

Small dataset - Less than 1k (Still depends on context)
Large dataset - Depends on context

Choose Logistic Regression when

The data is approximately linear; large dataset; need to explain coefficients to stakeholders.

Quick Reference — KNN vs Logistic Regression

Property	KNN	Logistic Regression
Training cost	None (lazy learner)	Iterative optimisation
Prediction cost	$O(N)$ — slow for large data	$O(d)$ — constant, fast
Decision boundary	Flexible / non-linear	Always linear (hyperplane)
Feature scaling	Critical — must standardise	Helpful but not critical
Many features (>20)	Suffers — curse of dimensionality	Handles well with regularisation
Interpretability	Low — "because the neighbours said so"	High — coefficients explain effects
Outputs probability	Approximately (vote fractions only)	Yes — via the sigmoid $\sigma(z)$

Neither algorithm is universally better. **Test both** and let the validation metrics decide.

KNN in scikit-learn: three focused stages.

The code follows the same structure as Lecture 04 — split, scale, train, evaluate — with one critical addition: feature scaling first.

Stage 1 — Split Data & Scale Features

- ▶ Split: 80% training, 20% test (same as Lecture 04)
- ▶ `fit_transform` on train — *learns μ and σ*
- ▶ `transform` on test — *applies the same scale*

Never call `fit` on the test set. Fitting on test data leaks information about test samples into the model.

```
knn_stage1.py

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# — Stage 1a: Split (same as Lecture 04) —
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# — Stage 1b: Scale features —
# fit_transform on TRAIN → learns mean and std
# transform on TEST → applies same scale
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

Stage 2 — Find the Best k by Validation

- ▶ Loop through **odd** k : 1, 3, 5 ... 25
- ▶ For each k : fit on train, score on test
- ▶ Track the k that gives the **highest accuracy**

💡 This is the validation curve from slide 16 translated into code. The loop *measures* instead of guessing.

```
knn_stage2.py

from sklearn.neighbors import KNeighborsClassifier

# — Stage 2: Loop over odd k values —
best_k = 1
best_acc = 0.0

for k in range(1, 26, 2): # step=2 → odd values only
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_sc, y_train)

    acc = knn.score(X_test_sc, y_test)
    print(f"k={k:>2}  accuracy={acc:.3f}")

    if acc > best_acc:
        best_k, best_acc = k, acc

print(f"\nBest k = {best_k}  (accuracy = {best_acc:.3f})")
```

Stage 3 — Train Final Model & Evaluate

- ▶ Re-fit with the best k found in Stage 2
- ▶ Predict on the test set
- ▶ Use same metrics as Lecture 04:
accuracy, precision, recall, F1, confusion matrix

i `classification_report` prints all four metrics in one call — use it as the standard evaluation tool for any classifier.

```
knn_stage3.py

from sklearn.metrics import (
    classification_report,
    confusion_matrix
)

# — Stage 3: Final model with best k —
final_knn = KNeighborsClassifier(n_neighbors=best_k)
final_knn.fit(X_train_sc, y_train)

y_pred = final_knn.predict(X_test_sc)

# Evaluation (same as Lecture 04)
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:\n", cm)
```

Six things to walk away with

- 01** KNN classifies by **majority vote** of the k nearest neighbours in feature space.
- 02** **Scale your features** — or the widest-range feature dominates every distance calculation.
- 03** k too small = **overfitting**. k too large = **underfitting**. Choose k by measuring on a validation set.
- 04** Use a **validation curve** to pick the best k — not a rule of thumb.
- 05** KNN is a **lazy learner**: no training cost, but $O(N)$ prediction cost — slow on large datasets.
- 06** The **curse of dimensionality**: adding irrelevant features makes distances meaningless in high- d space.